



Princípios de Segurança da Informação e DevSecOps

Integrando Confidencialidade, Integridade e Disponibilidade ao Ciclo de Desenvolvimento de Software.

INSTITUIÇÃO EXECUTORA

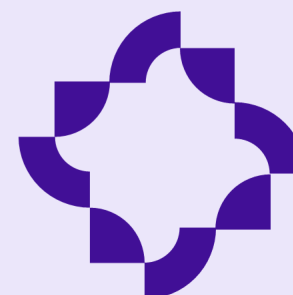


COORDENADORA



APOIO

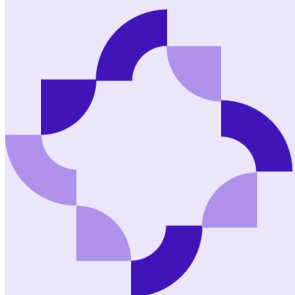
MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Sumário

- A Ascensão do DevSecOps
- Pilares da Segurança: CIA e Adicionais
- Security by Design e Responsabilidade Compartilhada
- Segurança como Código e Testes Automatizados
- Desafios e Otimização da Gestão de Vulnerabilidades



INSTITUIÇÃO EXECUTORA

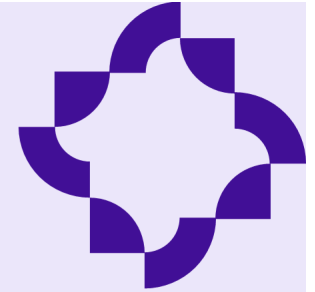


COORDENADORA



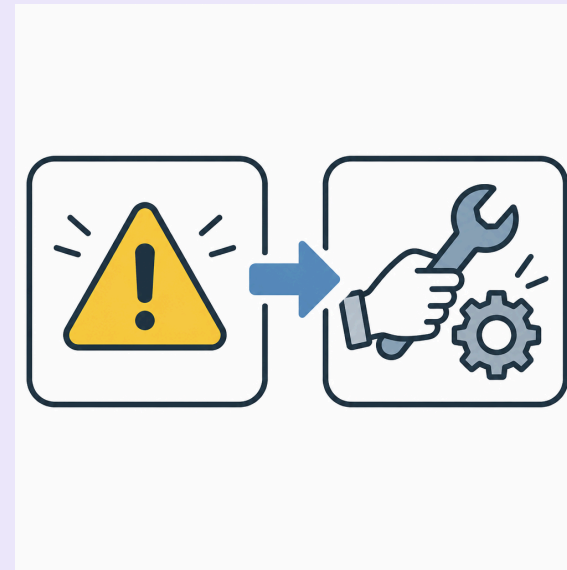
APOIO





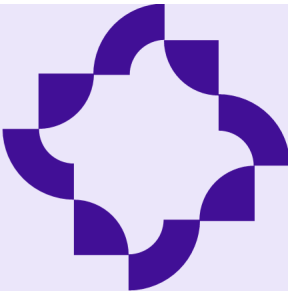
A Ascensão do DevSecOps

A segurança da informação, antes uma barreira final, agora é uma prática integrada. O modelo **DevSecOps** incorpora a segurança no Ciclo de Vida de Desenvolvimento de Software (SDLC) desde o início, equilibrando a agilidade do DevOps com o rigor da segurança. Promove a mentalidade de que 'todos são responsáveis pela segurança', mitigando riscos precocemente e reduzindo custos de correção de vulnerabilidades em produção.



Sinal de alerta e ferramentas: segurança integrada no SDLC.



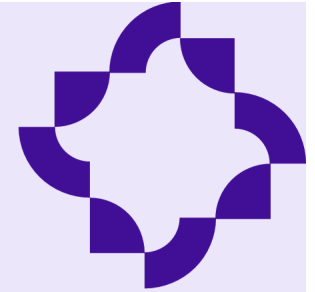


DevOps vs. DevSecOps

Esta tabela compara as principais características dos paradigmas DevOps tradicional e DevSecOps, destacando a integração proativa da segurança.

Característica	DevOps Tradicional	DevSecOps
Foco Principal	Velocidade e Agilidade	Velocidade com Segurança
Segurança	Reativa (fim do ciclo)	Proativa (Shift-Left)
Responsabilidade	Silos de TI/Ops	Responsabilidade Compartilhada
Feedback	Performance e Bugs	Performance, Bugs e Vulnerabilidades
Automação	Deploy e Infraestrutura	Segurança como Código (SaC)



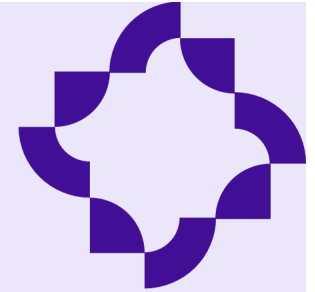


A Tríade CIA da Segurança



Escudo de cibersegurança: proteção de dados e redes globais.

- **Confidencialidade:** Garante que o acesso à informação seja restrito apenas a indivíduos e processos autorizados, frequentemente por meio de criptografia de dados em repouso e em trânsito.
- **Integridade:** Assegura que os dados permaneçam precisos e confiáveis, protegendo-os contra alterações não autorizadas ou acidentais.
- **Disponibilidade:** Garante que os sistemas e os dados críticos estejam acessíveis aos usuários legítimos sempre que necessário, mitigando riscos como ataques de negação de serviço.



Autenticidade e Não-Repúdio



Veracidade da Identidade

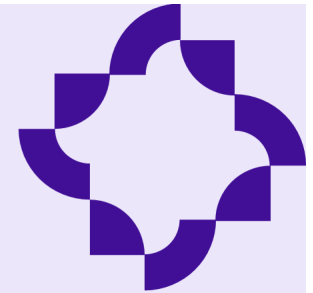
A **Autenticidade** é a garantia de que a identidade do usuário ou sistema que originou a informação é *verídica*. É um princípio essencial para o controle de acesso em APIs e ambientes de nuvem, assegurando que apenas entidades legítimas interajam com os recursos.



Provas Irrefutáveis de Ações

O **Não-Repúdio** é a capacidade de garantir que um autor não possa negar a autoria de uma transação ou alteração no sistema. No contexto do DevOps, este é um princípio vital para a *auditabilidade* de commits e mudanças em infraestrutura via código (IaC), promovendo rastreabilidade.





Conceito Security by Design

Projeto Desde o Início

Security by Design significa projetar sistemas para serem seguros desde o primeiro rascunho da arquitetura, em vez de adicionar camadas de segurança posteriormente. Esta abordagem *proativa* é crucial para criar sistemas robustos e resilientes.

Princípios Essenciais

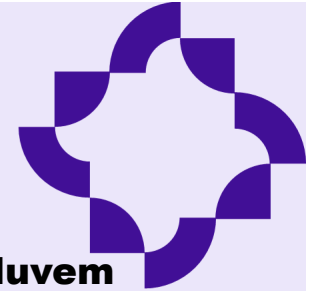
1. **Minimização da Superfície de Ataque:** Reduzir o número de pontos de entrada vulneráveis em um sistema, diminuindo as oportunidades para exploração. 2. **Princípio do Privilégio Mínimo:** Garantir que cada usuário ou processo tenha apenas o acesso estritamente necessário para realizar sua função, limitando o potencial de dano.

Defesa em Profundidade

- • • • 3. **Defesa em Profundidade:** Implementar múltiplas camadas de controle de segurança redundantes, de modo que a falha de uma não comprometa a integridade ou confidencialidade do sistema inteiro. Este conceito cria barreiras sucessivas contra ameaças.



Escudo de segurança digital protegendo dispositivos e dados em rede.



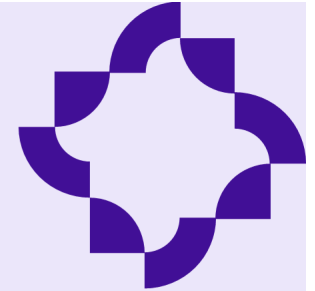
Provedor: Segurança 'DA' Nuvem

Responsável pela segurança da infraestrutura fundamental que compõe a nuvem. Isso engloba a proteção do **hardware físico**, da **infraestrutura de rede**, dos sistemas de virtualização e das instalações físicas que abrigam os serviços. O provedor garante a resiliência e a operacionalidade dos serviços subjacentes.

Consumidor: Segurança 'NA' Nuvem

Responsável pela segurança dos elementos alocados e configurados *dentro* do ambiente de nuvem. Inclui a proteção dos **dados do cliente**, o gerenciamento de identidade e acesso (IAM), a configuração de firewalls e redes virtuais, e a segurança do código das aplicações implementadas.

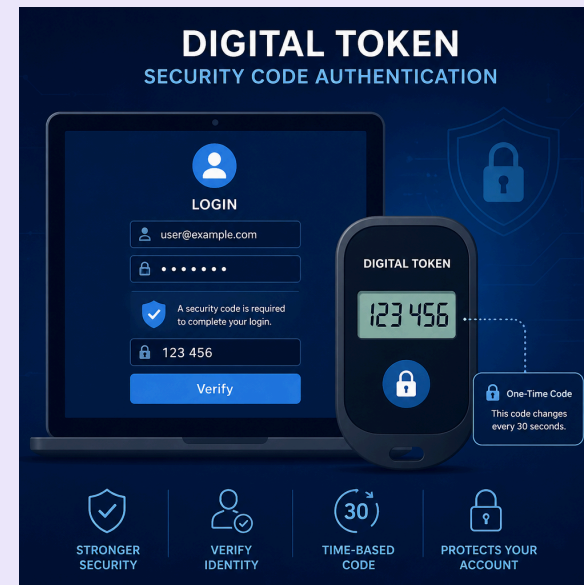




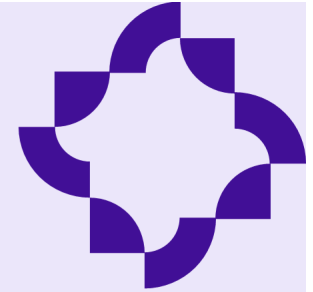
Segurança como Código (SaC)

A **Segurança como Código** (SaC, *Security as Code*) representa a aplicação de automação para implementar e validar políticas de segurança, transformando requisitos de conformidade em scripts executáveis que verificam continuamente o ambiente.

Políticas de segurança são tratadas como código, permitindo **versionamento**, rastreabilidade, revisões por pares e *rollback* rápido. Esta abordagem garante **escalabilidade**, acompanhando a velocidade de centenas de deploys diários sem intervenção manual constante.



Autenticação por token digital: segurança em código para login.



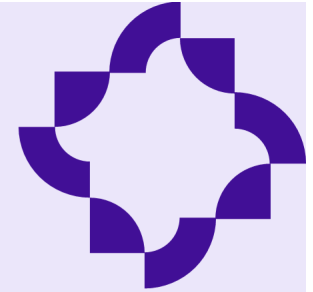
Análise Estática (SAST)

Analisa o código-fonte, byte code ou binários **sem executar** a aplicação. Identifica falhas lógicas e vulnerabilidades comuns (OWASP Top 10) no estágio de 'Code' ou 'Build' do ciclo de desenvolvimento, permitindo correções *proativas*.

Análise Dinâmica (DAST)

Testa a aplicação **em tempo de execução**, simulando ataques externos. Detecta vulnerabilidades que só aparecem quando o sistema está ativo, como falhas de autenticação de sessão ou configurações incorretas de servidor, revelando problemas *reativos*.





O Fluxo DevSecOps

1

Planejamento

Nesta fase, realiza-se a **Modelagem de Ameaças** (*Threat Modeling*) para identificar e documentar potenciais riscos de segurança desde o início do ciclo de vida do desenvolvimento.

2

Codificação Segura

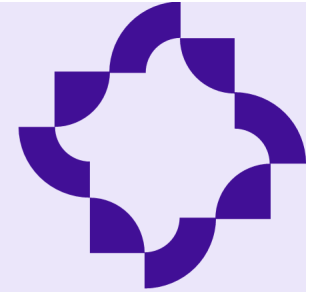
Durante a codificação, são utilizados **IDEs com plugins de segurança** para detecção precoce de vulnerabilidades e é feita a análise de dependências de código aberto (*Software Composition Analysis* - SCA) para gerenciar riscos de bibliotecas de terceiros.

3

Build & Análise SAST

No estágio de *build*, ferramentas de **Análise Estática de Segurança de Aplicações** (*Static Application Security Testing* - SAST) são executadas no código-fonte, e a integridade dos artefatos é verificada para garantir que nenhuma alteração maliciosa foi introduzida.





O Fluxo DevSecOps

4

Testes de Segurança (DAST)

Aqui, a automação de **testes funcionais de segurança** é implementada, incluindo a execução de *DAST (Dynamic Application Security Testing)* em ambientes de *staging* para identificar vulnerabilidades em tempo de execução.

5

Release & Deploy Seguro

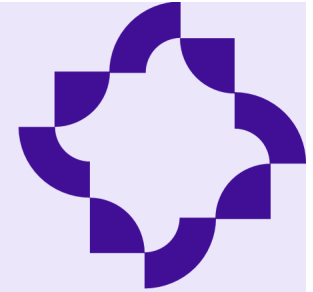
Antes da implantação, é crucial verificar as configurações da infraestrutura (*IaC Scan - Infrastructure as Code Scan*) e garantir que segredos criptográficos (chaves, senhas) sejam gerenciados e armazenados de forma segura, evitando vazamentos.

6

Operação & Monitoramento

Após a implantação, o **monitoramento contínuo** em produção é essencial. Sistemas de *logging* e alerta são configurados para permitir uma **resposta a incidentes** em tempo real, mitigando ameaças rapidamente.

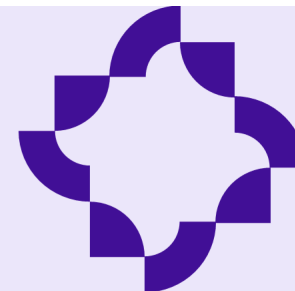




Escudo de cibersegurança protege redes e dados contra ameaças.

Gestão de Vulnerabilidades no DevSecOps

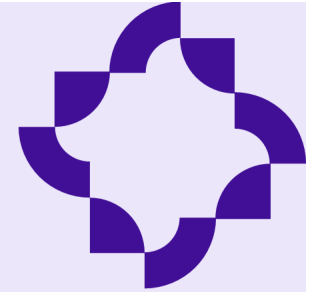
- **Priorização:** Métricas como o CVSS (Common Vulnerability Scoring System) são cruciais para classificar a gravidade das vulnerabilidades, direcionando a atenção para as mais críticas.
- **Interrupção do Build:** Vulnerabilidades de 'Crítico' ou 'Alto' nível devem interromper automaticamente o pipeline de deploy (Breaking the Build), garantindo que falhas graves não cheguem à produção.
- **Feedback Direto:** A automação deve fornecer feedback imediato e acionável aos desenvolvedores, incluindo sugestões para correção de código ou atualização de bibliotecas.



Escudo de cibersegurança protege redes e dados contra ameaças.

Gestão de Vulnerabilidades no DevSecOps

- **Redução do MTTR:** O objetivo primário é minimizar o Tempo Médio de Remediação (Mean Time to Remediation) através de processos automatizados e fluxo de trabalho eficiente.



Desafios na Implementação

Resistência Cultural

Desenvolvedores podem ver a segurança como um bloqueio à velocidade. É necessário fomentar uma cultura de empatia e colaboração para superar essa barreira, integrando a segurança de forma transparente no ciclo de desenvolvimento.

Falsos Positivos

Ferramentas automáticas podem gerar alertas irrelevantes que sobrecarregam as equipes, causando fadiga de alerta. O ajuste fino (*tuning*) e a calibração dessas ferramentas são vitais para garantir a relevância e priorização das descobertas.

Escassez de Talentos



A implementação eficaz de DevSecOps exige profissionais que entendam tanto de desenvolvimento quanto de segurança cibernética avançada. A falta desses talentos especializados representa um desafio significativo para as organizações.



Discuta com o colega ao lado

Em um cenário de correção crítica, como a equipe de DevSecOps deve proceder para garantir a agilidade sem comprometer a segurança, dado que o scan de segurança completo excede o tempo limite operacional?

Considere estratégias para equilibrar **disponibilidade** e **integridade** neste contexto de pressão temporal.



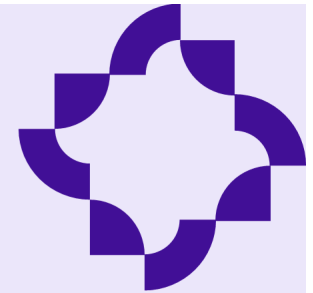


Verdadeiro ou falso?

No modelo de responsabilidade compartilhada da nuvem, o provedor é o único responsável por garantir que o código da aplicação do cliente não tenha vulnerabilidades de SQL Injection.

← Falso

Verdadeiro →



× Falso

No modelo de responsabilidade compartilhada da nuvem, o provedor é o único responsável por garantir que o código da aplicação do cliente não tenha vulnerabilidades de SQL Injection.

Explicação

No modelo de responsabilidade compartilhada, o cliente é o responsável pela segurança *na* nuvem, incluindo a segurança do seu código e aplicações.



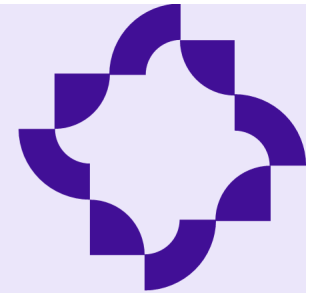


Verdadeiro ou falso?

O conceito de 'Shift-Left' refere-se ao deslocamento das práticas de segurança para as fases iniciais do desenvolvimento, como o planejamento e a codificação.

← Falso

Verdadeiro →



✓ Verdadeiro

O conceito de 'Shift-Left' refere-se ao deslocamento das práticas de segurança para as fases iniciais do desenvolvimento, como o planejamento e a codificação.

Explicação

O 'Shift-Left' visa integrar a segurança desde o início do ciclo de vida de desenvolvimento de software (SDLC) para identificar e corrigir vulnerabilidades precocemente.



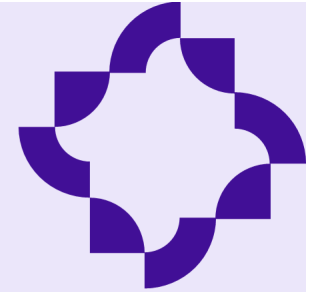


Verdadeiro ou falso?

SAST e DAST são ferramentas redundantes; a execução de apenas uma delas é suficiente para garantir a segurança completa de uma aplicação web.

← Falso

Verdadeiro →



× Falso

SAST e DAST são ferramentas redundantes; a execução de apenas uma delas é suficiente para garantir a segurança completa de uma aplicação web.

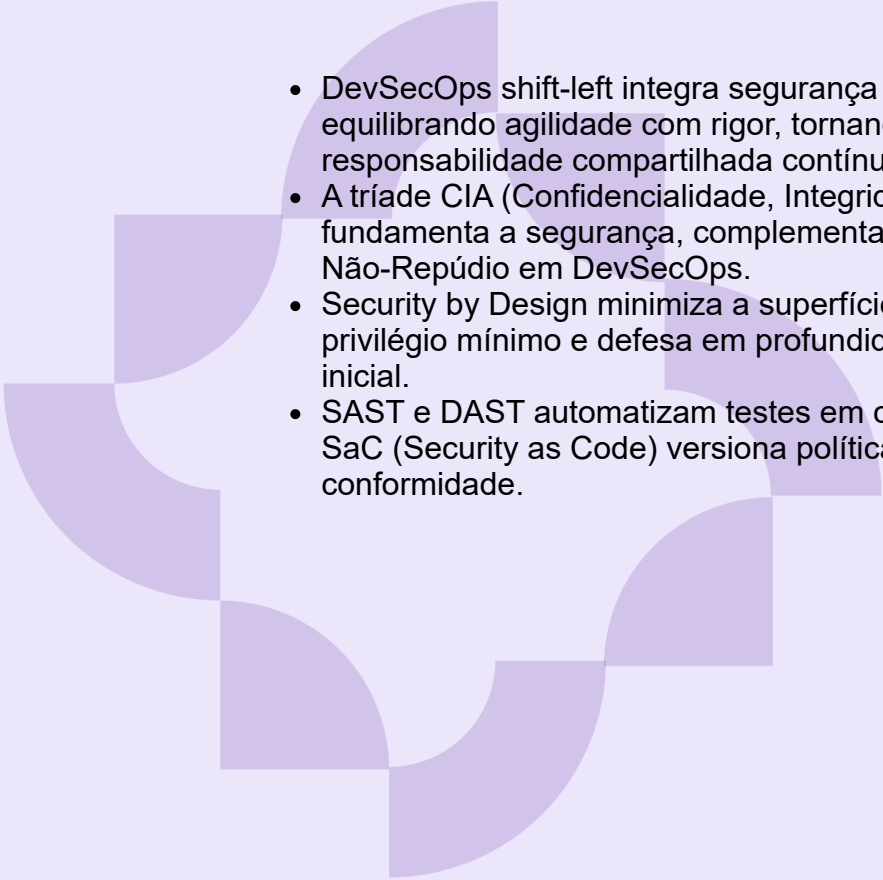
Explicação

SAST (Static Application Security Testing) e DAST (Dynamic Application Security Testing) são ferramentas complementares, cobrindo diferentes aspectos da segurança de aplicações e não são redundantes.





Conclusão

- 
- DevSecOps shift-left integra segurança proativamente, equilibrando agilidade com rigor, tornando a segurança uma responsabilidade compartilhada contínua.
 - A tríade CIA (Confidencialidade, Integridade, Disponibilidade) fundamenta a segurança, complementada por Autenticidade e Não-Repúdio em DevSecOps.
 - Security by Design minimiza a superfície de ataque e aplica privilégio mínimo e defesa em profundidade desde o projeto inicial.
 - SAST e DAST automatizam testes em código e runtime, enquanto SaC (Security as Code) versiona políticas para escalabilidade e conformidade.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

